Employee Management System using JSON Server and Postman

Objective: To configure the Fake Rest API using JSON Server and run the db.json file on a port number 3000 to manage Employee Details as given below requirements**.**

Software Requirements

- Node.js
- npm
- JSON Server
- Postman

Step 1: Create the *dbs.json* File

- Created a *dbs.json* file containing employee records with attributes such as Employee ID, Name, Department, Designation, Salary, and Email. This file acts as a temporary database for the JSON Server.
- **Screenshot 1:** *dbs.json File*

File Edit View


```
{
  "employees": [
    {
      "eid": 101,
      "ename": "John",
      "email": "john@abc.com",
      "mobile": 1039417,
      "id": "3vsRNQQlMTI"
    },
    {
      "eid": 102,
      "ename": "Jane",
      "email": "jane@abc.com",
      "mobile": 1047654,
      "id": "2GrMKwRyISU"
    },
    {
      "eid": 104,
      "ename": "Monk",
      "email": "monk@abc.com",
      "mobile": 1029687,
      "id": "9RJ75aKmZEY"
    }
  ],
  "$schema": "./node_modules/json-server/schema.json"
}
```

Step 2: Start JSON Server

Opened the terminal in the project folder and executed the following command to start the JSON Server on port 3000.

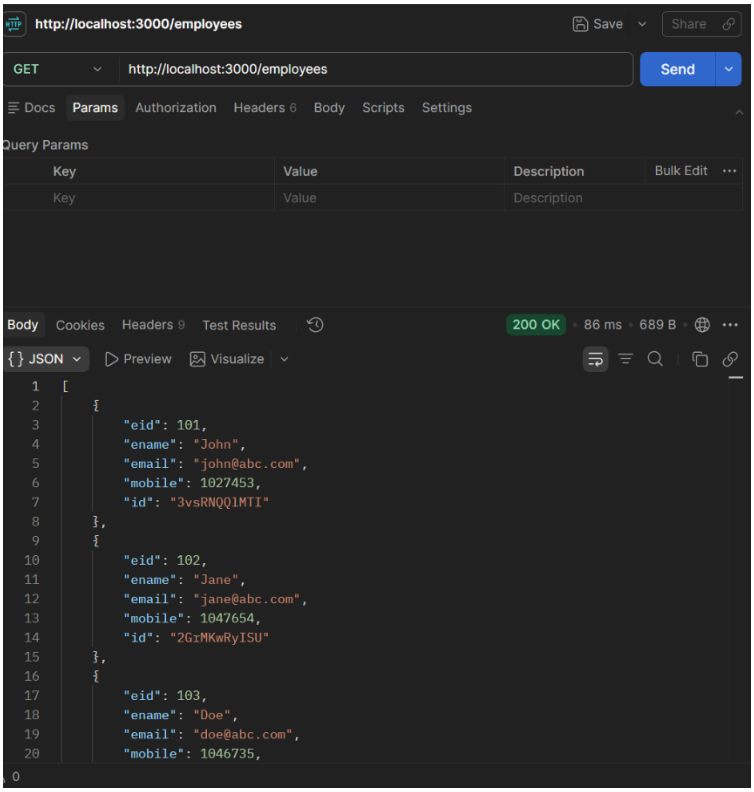
Screenshot 2: *JSON Server Running*

```
PS C:\StudentDataRestAPI> npx json-server dbs.json --port 3000
npm notice run npx
npm notice run json-server dbs.json --port 3000
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching dbs.json...

♡( ◡‿◡ )

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/employees
```

Step 3: Test GET API (Retrieve All Employees)

Opened Postman and sent a **GET** request to retrieve all employee records stored in the JSON file.

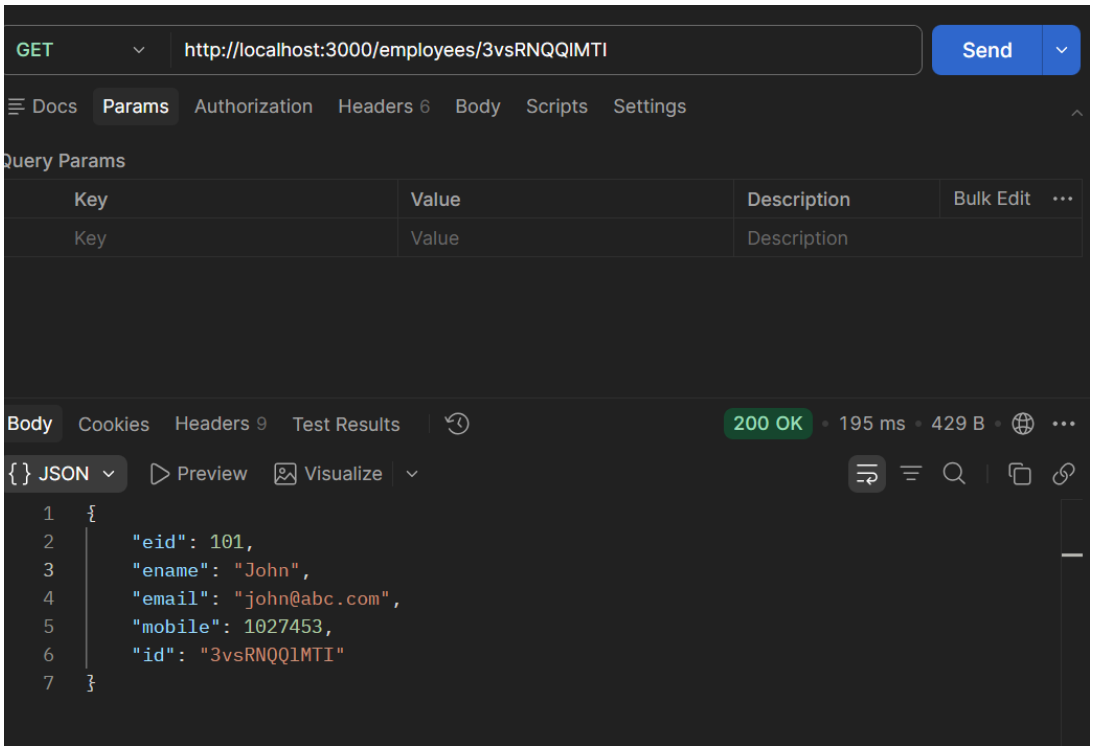
Screenshot 3: *GET All Employees*



Step 4: Test GET API (Retrieve Employee by ID)

Sent a GET request with a specific employee ID to retrieve the details of a single employee.

Screenshot 4: *GET Employee by ID*

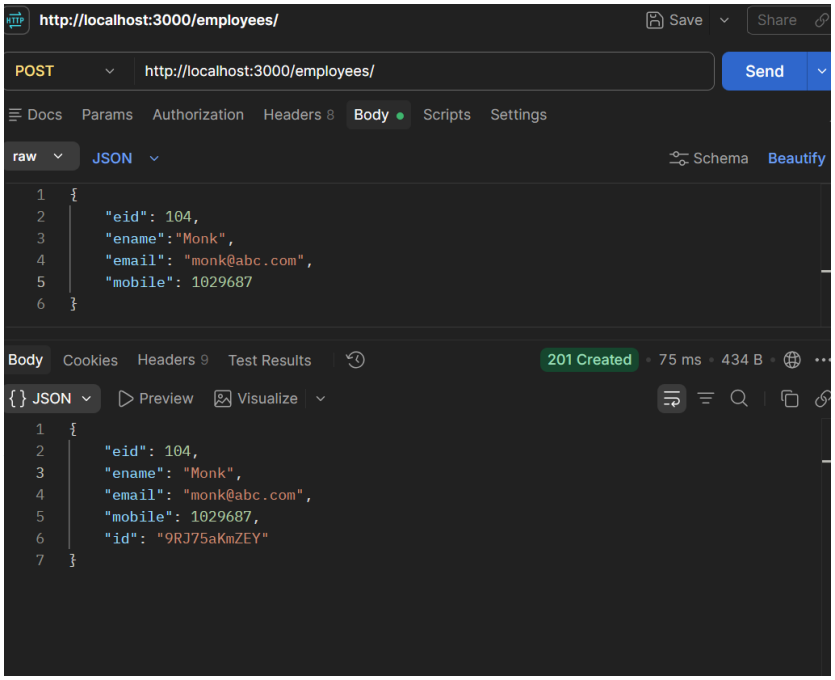


Step 5: Test POST API (Add New Employee)

Changed the request method to **POST** and added a new employee record by sending JSON data in the request body.

The employee was successfully inserted into the database.

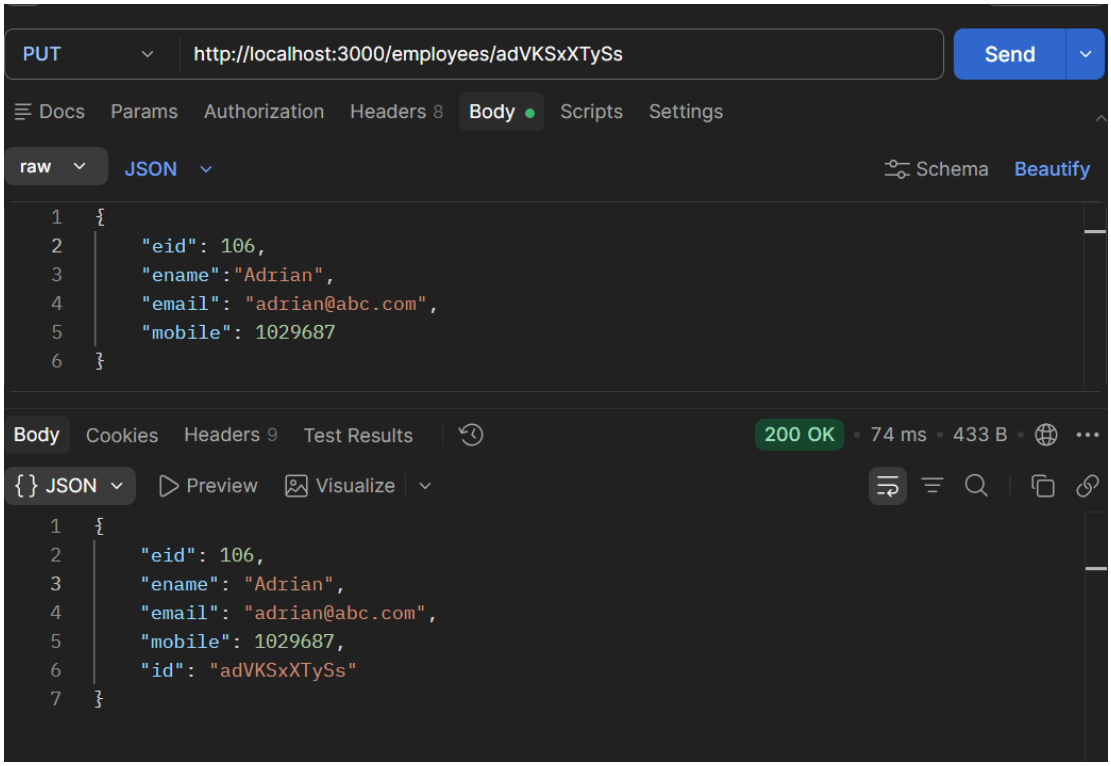
Screenshot 5: *POST Employee*

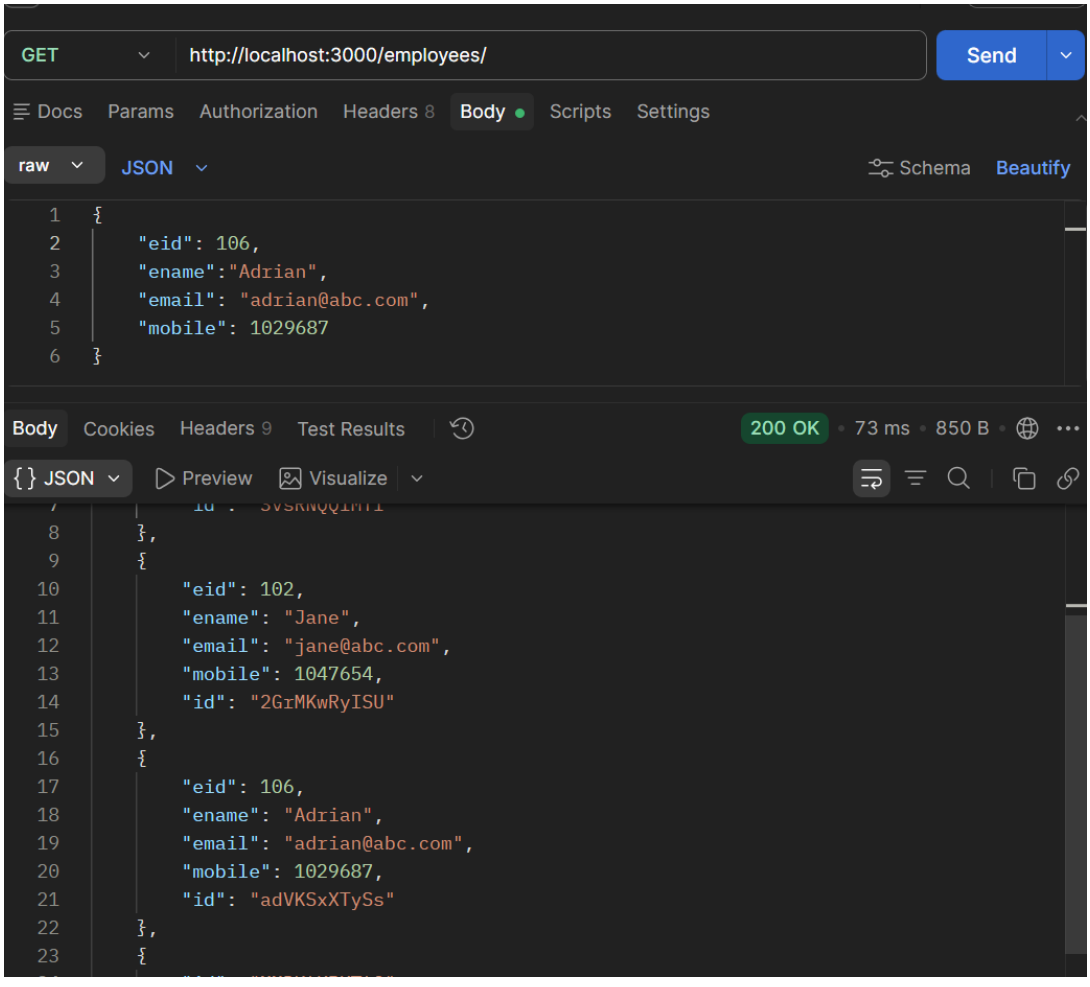


Step 6: Test PUT API (Update Employee Details)

Used the **PUT** request to update the complete details of an existing employee. The entire employee object was replaced with the updated information.

Screenshot 6: *PUT Employee*



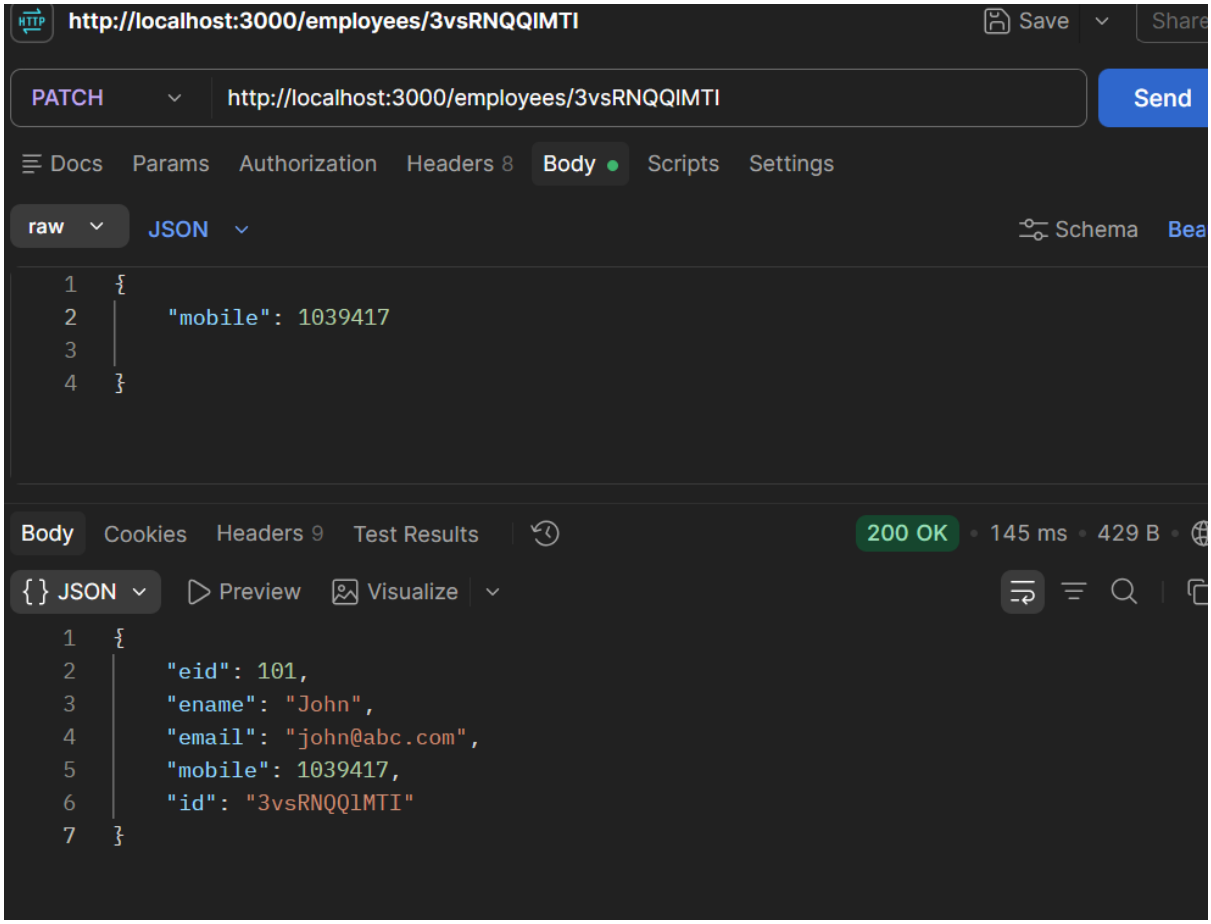


Step 7: Test PATCH API (Update Specific Field)

Used the **PATCH** request to modify only a specific field (for example, Salary or Department) without changing the remaining employee details.

Screenshot 7: *PATCH Employee*

(Insert Screenshot Here)

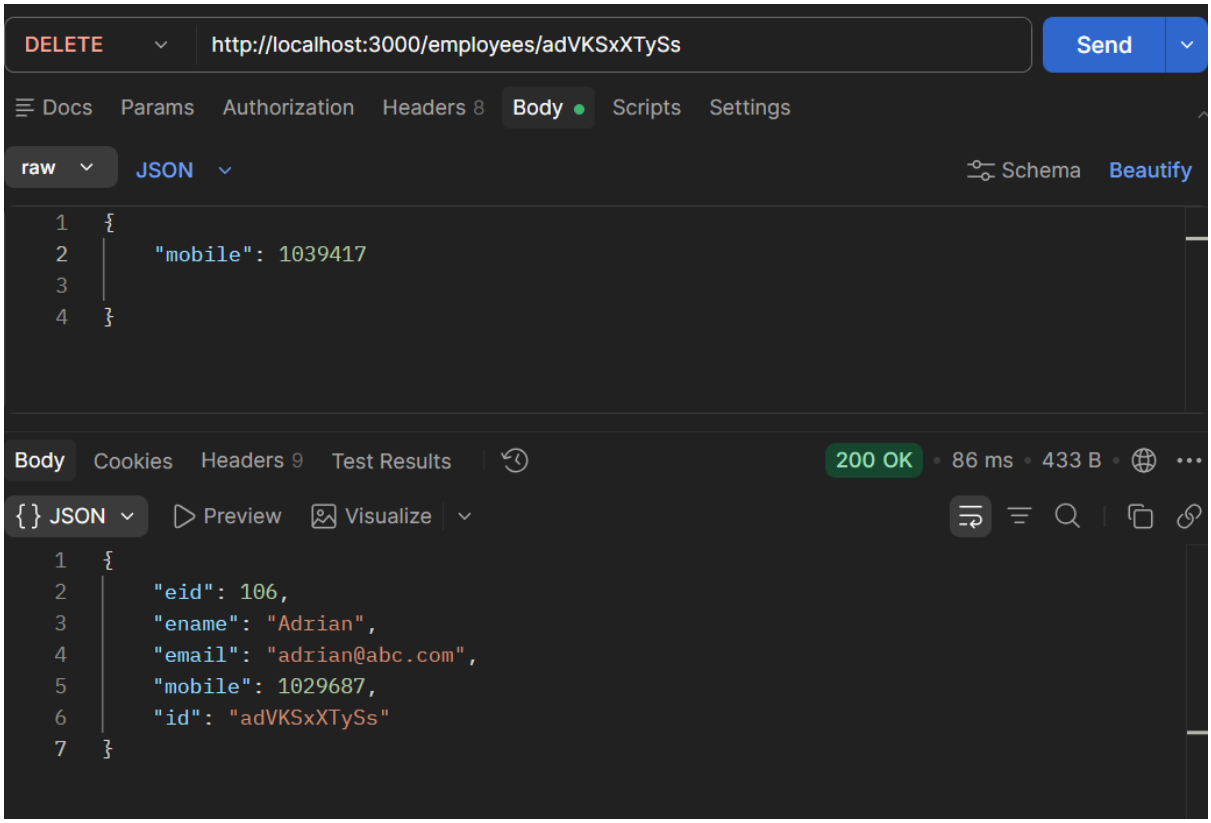


Step 8: Test DELETE API (Delete Employee)

Sent a **DELETE** request to remove an employee record from the database using the employee ID.

The employee record was deleted successfully.

Screenshot 8: *DELETE Employee*



Step 9: Verify Deletion

Performed another **GET** request to verify that the deleted employee no longer exists in the employee list.

Screenshot 9: *Verification after DELETE*